

Actividad Corta de Laboratorio: De ADO.NET Tradicional a la Automatización con EF Core

Duración estimada: 35 minutos

Modalidad: Individual

Entregable: Un documento corto en Markdown (Actividad_ORM.md) con las respuestas teóricas y los dos fragmentos de código solicitados.

Parte 1: Diagnóstico Técnico y Brecha de Impedancia

Con base en el material de las **Sesiones 16, 17 y 18** del laboratorio de IPC 2, responde de forma sintética a las siguientes preguntas:

1. **La Brecha de Impedancia:** Explica brevemente la diferencia conceptual entre el *Dominio de Objetos (C#)* y el *Dominio Relacional (SQL)*, completando las siguientes equivalencias según el mapeador :
 - Clase Clásica (POCO) -> Mapea a -> _____
 - Propiedad/Atributo -> Mapea a -> _____
 - Instancia de Objeto -> Mapea a -> _____
2. **Mitigación de Vulnerabilidades:** En ADO.NET tradicional, concatenar variables directamente en un string SQL genera un riesgo crítico de **Inyección SQL** . Explica qué propiedad nativa de EF Core previene este fallo de seguridad de forma automatizada y qué comando equivalente usabas en ADO.NET para mitigarlo .
3. **Optimización de Infraestructura:** Explica brevemente por qué el uso del método `.AsNoTracking()` en consultas LINQ de EF Core es una muestra de *solidaridad computacional* con el hardware compartido de la universidad .

Parte 2: Desafío de Refactorización de Código

Imagina que estás trabajando en el *Sistema de Control de Catedráticos* de la USAC y te han pedido migrar un viejo módulo de software de bajo nivel hacia el ORM oficial de Microsoft .

Paso 1: Analiza el código obsoleto en ADO.NET (Lectura de 2 minutos)

Este código abre una conexión física, usa un bloque using para liberar recursos y parametriza para evitar ataques, pero requiere mapeo manual fila por fila :

```
// CÓDIGO VIEJO A REFACTORIZAR (ADO.NET)
public ObtenerCatedraticosViejos string
{
    var lista = new List<Catedratico>();
    string query = "SELECT Id, Nombre FROM Tbl_Catedraticos WHERE Nombre LIKE @filtro";
```

```

using (SqlConnection conn = new SqlConnection(connectionString))
{
    using (SqlCommand cmd = new SqlCommand(query, conn))
    {
        cmd.Parameters.AddWithValue("@filtro", "Ing.%");
        conn.Open();
        using (SqlDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                lista.Add(new Catedratico {
                    Id = reader.GetInt32(0),
                    Nombre = reader.GetString(1)
                });
            }
        }
    }
}
return lista;
}

```

Paso 2: Escribe la versión moderna con EF Core (23 minutos)

En tu entregable, escribe los siguientes dos fragmentos de código utilizando las convenciones de **Entity Framework Core**:

1. **El Contexto (DbContext)**: Define la clase UnidadAcademicaContext que herede de DbContext e incluya el DbSet correspondiente para poder mapear la tabla de catedráticos .
2. **La Consulta LINQ**: Escribe un método moderno en C# que reciba tu instancia de _context y devuelva la lista de catedráticos cuyo nombre empiece con "Ing." .
 - *Requisito de rendimiento obligatorio*: Ya que este flujo es exclusivo de lectura para renderizar una lista, debes apagar el rastreador de cambios para liberar memoria RAM crítica en el servidor .

Parte 3: Referencias Bibliográficas (Obligatorio)

Al final de tu documento, debes incluir la base bibliográfica formal utilizada para resolver la prueba:

- Facultad de Ingeniería, USAC. (2026). *Sesión 17: Conectividad con SQL Server. Acceso Estructurado a Datos mediante C# y ADO.NET*. Laboratorio de Introducción a la Programación y Computación 2. Guatemala.
- Facultad de Ingeniería, USAC. (2026). *Sesión 18: Mapeo de Objetos Relacionales. Persistencia Automatizada con Entity Framework Core*. Laboratorio de Introducción a la Programación y Computación 2. Guatemala.